Kevin Küster

Zusammenfassung: Rechnerstrukturen

Rechnerstrukturen

Pipelining & Scoreboards

Pipelining

Pipelining (Fließband-Prinzip) zerlegt Ausführung einer Instruktion in mehrere Phasen, die nacheinander durchlaufen werden → Gleichzeitige Verarbeitung
↳ Idealfall: pro Takt eine Instruktion fertig

▷ Pipeline-Phasen: Fetch Instruction (FI), Decode Instruction (DI), Fetch Operand (FO), Execute Instruction (EI), Write Operand (WO)

Hazards

▷ Strukturelle Hazards: Zwei Instruktionen brauchen in derselben Phase dieselbe Hardware-Ressource (z.B. nur ein Speicherport für Befehl- & Datenzugriff) ⇒ Ressource replizieren oder Pipeline anhalten (Stall)

▷ Datenhazard: Eine Instruktion hängt vom Ergebnis einer früheren ab
↳ Read after Write (RAW): echte Abhängigkeit
↳ Write after Write (WAW): Output-Konflikt
↳ Write after Read (WAR): Anti-Konflikt

▷ Kontroll-Hazards: Bedingter Sprung verändert den Programmzähler. Bereits in die Pipeline geladene Folge-Instruktionen können obsolet sein
⇒ Branch Prediction, Delay Slots

Beispiel

(i) RAW: I1: ADD R2, R3, R1 ; R1 := R2 + R3
I2: SUB R5, R1, R4 ; R4 := R1 - R5 ← liest R1, das I1 schreibt
⇒ I2 muss auf Abschluss von R1 warten ⇒ Forwarding/Bypassing

(ii) WAW: I1: ADD R2, R3, R1 ; R1 := R2 + R3
I2: MUL R6, R7, R1 ; R1 := R6 · R7 ← schreibt nochmal R1
⇒ Endgültiges R1 muss von I2 sein; Bei out-of-order Ausführung problematisch ⇒ Register-Renaming

(iii) WAR: I1: ADD R2, R3, R1 ; R1 := R2 + R3
I2: SUB R4, R5, R3 ; schreibt R3 ← Darf nicht vor I1 Read passieren
⇒ Register Renaming

Scoreboard

Hardware-Buchführungseinheit im Leitwerk, die mehrere Befehle parallel ausführen lässt → Erkennt Datenabhängigkeiten & Ressourcen-Konflikte und verzögert betroffene Befehle; Vier Phasen pro Befehl:

(i) Issue: Befehl erhält Funktionseinheit (FU) - wenn frei und kein WAW-Konflikt

(ii) Read Operands: Operanden lesen, sofern keine RAW-Abhängigkeiten besteht

(iii) Execute: FU rechnet die Operation aus; Dauer abhängig von Operation (Add ≈ 2, Mul ≈ 3, Div ≈ 10 Zyklen)

(iv) Write Result: Ergebnis ins Register schreiben → keine WAR-Abhängigkeiten

▷ Forwarding: Ergebnis einer Instruktion wird direkt aus dem Latch zwischen Pipeline-Stufen an folgende Instruktion weitergegeben
↳ Eliminiert die meisten RAW-Stalls

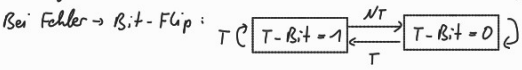
▷ Register-Renaming: Umbenennung der logischen Register, sodass mehrfach verwendete Namen physisch unterschieden werden.
↳ Eliminiert WAW & WAR

Branch Prediction

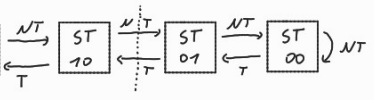
In moderner Pipeline ist erst nach mehreren Zyklen klar, ob ein bedingter Sprung genommen wird oder nicht; In dieser Zeit werden bereits weitere Instruktionen geladen. Falls falsche Ins. geladen: Verwerfen (Pipeline-Flush)

▷ Statische branch prediction: Vorhersage zur Compile-Zeit, feste Regel
↳ Strategien:
- never taken → Immer raten, dass Sprung nicht genommen wird ⇒ ~30% - 40%
- always taken → Immer raten, dass Sprung genommen wird ⇒ ~60% - 65%
- Backward taken forward not taken (BTFNT): Rückwärtssprung (z.B. Schleife) → taken; Vorwärtssprung (z.B. Verzweigung) → Not taken ⇒ ~70% - 75%
- Opcode-abhängig: Vorhersage auf Basis des Sprungbefehl-Typs

▷ dynamische branch prediction: Vorhersage zur Laufzeit basierend auf vergangener Historie. Branch History Table speichert für jede Branchinstruktion Zustandsautomaten.
↳ Strategien:
- 1-Bit-History: Pro branch-Instruktion 1 Bit, das letzten Ausgang merkt; Bei Fehler → Bit-Flip:



- 2-Bit-History: Jede Umschwenkung erhält „zweite Chance" → 2-Bit-Counter → 4 Zustände
ST: Strongly Taken; WT: Weakly Taken; WNT: "Not"; SNT: "Not"



⇒ Branch-History-Table (BHT): Hardware-Datenstruktur, die Vorhersage-Bits zu den Branches speichert:

Branch-Adr.	PC-Wert der Branch-Instruktion
Sprungziel (Target)	Adresse des Branch-Ziels; Wird bei taken ohne neue Berechnung verwendet
Status-Bits (History)	1-Bit oder 2-Bit Zustands-Wert

▷ Delayed Branch: Befehl der direkt nach einem Sprung-Befehl immer ausgeführt wird ⇒ Sinnvolle Belegung:
↳ Vorgänger-Befehl, Befehl vom Sprungziel, Befehl aus else-Zweig

Amdahl & Parallelisierung

Gesetz von Amdahl

Gibt eine obere Schranke Schranke für den Speedup, der durch Parallelisierung erreicht werden kann. → Sequentieller Anteil f als Engpass

→ Sequentieller Anteil: Bruchteil eines Programms, der nicht parallelisiert werden kann (Initialisierung, I/O, Synchronisation, kritische Sektoren)

▷ Amdahl-Formel:

$$T(n) = f \cdot T(1) + \frac{(1-f) \cdot T(1)}{n}$$

→ T(n): Ausführungszeit n Prozessoren

→ T(1): Ausführungszeit auf 1 Prozessor (=100% normiert) = 1

→ f: Sequentieller Anteil ⇒ $f \cdot T(1)$ ⇒ $f = \alpha - 1$; $0 \le f \le 1$

→ Paralleler Anteil: $(1-f) \cdot T(1) = \alpha = 1 - f$ → Verteilt auf n Prozessoren

▷ Speedup (Beschleunigungsfaktor): $S(n) = \frac{T(1)}{T(n)} = \frac{1}{f + \frac{(1-f)}{n}} = \frac{n}{1 + f \cdot (n-1)}$

→ S(n): Beschleunigungsfaktor bei n Prozessoren

⇒ Randfälle: n=1: S(1)=1 (kein Speedup); (keine Beschl. möglich); $\lim_{n \to \infty}$: $S_{max} = \frac{1}{f}$ (Obere Schranke)

▷ Effizienz: Wie gut ist die durchschnittliche Auslastung pro Prozess

$$E(n) = \frac{S(n)}{n}$$; E=1 ≙ 100%, E<1: Nicht alle Prozessoren ausgelastet

Cache-Kohärenz

Garantiert, dass alle Prozessoren in einem shared-Memory-System konsistente Sicht auf gemeinsame Daten haben

▷ Snoopy: Alle Caches lauschen (snoop) auf gemeinsamem Bus; Schreibvorgänge per Broadcast. Einfach, skaliert schlecht. → MESI (Modified-Exclusive-Shared-Invalid → 4 Cache-Zustände)

▷ Directory-Based: Verzeichnis hält Information, welche Caches welche Lines haben; Nur betroffene Caches benachrichtigen. Skaliert besser für NUMA (Non-Uniform Memory Access - verteilter Speicher)

MESI

Modified (M)	Lokal modifiziert; Nur in diesem Cache; Hauptspeicher veraltet (dirty)
Exclusive (E)	Konsistent mit Hauptspeicher (clean); Nur in diesem Cache
Shared (S)	Konsistent mit Hauptspeicher; Kann in mehreren Caches sein
Invalid (I)	Keine Gültigen Daten

Cache-Line

Kleinste Übertragungseinheit zwischen Hauptspeicher und Cache

↳ zusammenhängender Speicherblock fester größe (meist 64 Byte

CPU-Operationen

▷ Cache-HIT: Eigener Prozessor will Operation auf Cache-Line ausführen (Lesen/Schreiben) → Cache-Line im eigenen Cache vorhanden (→ Gültige Zustände: M, E, S)

CPU: Zugriff auf Adr. 0x4000 → Cache: Liegt im Zustand E → HIT

▷ Cache-Miss: -"-. → Cache-Line nicht (oder im Zustand I) im eigenen Cache ⇒ Muss zuerst geladen werden

CPU: -"- → Cache: Kein Zugriff → MISS ⇒ Bus-Anfrage, ggf. Hauptspeicher

▷ Übergänge:

Symbol	pre-Bed.	Folgezustand	Bus-Aktion
Read-HIT	Eigene Line: M	bleibe in M	nichts
Read-HIT	-"-: E	-"- E	-"-
Read-HIT	-"-: S	-"- S	-"-
Read-MISS	Niemand hat Line	E	Read-Anforderung auf Bus → Lade aus Hauptspeicher
Read-MISS	Andere haben Line in S	S	Read-Anforderung → -"-
Read-MISS	Anderer hat Line in M	S	Read-Anforderung; Der M-Cache schreibt, eigener liest
Write-Hit	Eigene Line: M	bleibt M	nichts
Write-Hit	-"-: E	M	nichts
Write-Hit	-"-: S	M	Invalidate auf Bus → andere I
Write-Miss		M	RWITM (Read with Intent to modify) auf Bus → Line laden + Kopien invalidieren

▷ Operationen Snoop-Schicht: Falls anderer Prozessor lesen/schreiben will, wird Op auf Bus gelegt → Eigene Cache schnüffelt das (snooping)

→ Snoop Read: Anderer Prozessor will lesen → Eigene Cache prüfen:

Eigener Zustand	Reaktion	Folgezustand
I	nichts	bleibt I
S	„ich habs' auch" signalisieren	bleibt S
E	„ich halte exclusive, jetzt teilen"	E → S
M	Dirty Line Copyback + signalisieren	M → S

Bei M: Da eigene Modifikation noch nicht im Hauptspeicher → Zurückschreiben (Copyback) bevor teilen

→ Snoop Write (= RWITM von Außen): Anderer Prozessor will schreiben → Cache:

Eigener Zustand	Reaktion	Folgezustand
I	nichts	bleibt I
S	Eigene Kopie wird ungültig	S → I
E	-"-; Kein Copyback → Hauptspeicher war aktuell	E → I
M	Dirty Line Copyback + eigene Kopie ungültig	M → I

Multithreading & Superskalarität

Superskalar

Prozessor, der pro Takt mehr als eine Instruktion ausführen kann; Dazu sind mehrere Funktionseinheiten parallel aktiv (z.B. mehrere ALUs)

=> Instructions per Cycle (IPC) > 1 möglich

Very long Instruction Word (VLIW)

Sehr lange Befehlsworte, die mehrere Operationen kodieren; Compiler entscheidet zur Übersetzungszeit, welche Operationen parallel ausgeführt werden sollen;

↳ pro: einfache Hardware (kein Out-Of-Order, kein Reorder Buffer); deterministisches Timing

↳ contra: Compiler trägt die Komplexität; Code-Größe wächst (NoOperation-Instruktion für unbelegte Slots); Abhängig von Speicher-Latenz, die Compiler nicht mehr kennt

Multithreading - Varianten

- Fine-Grained Multithreading (FMT): In jedem Takt wird ein anderer Thread ausgeführt; Latenz-Stalls eines Threads werden durch andere Threads überbrückt
- Coarse-Grained Multithreading (CMT)/Block-Multithreading: Ein Thread läuft so lange, bis er auf eine Latenz wartet (z.B. Cache-Miss); Dann wechselt Prozessor auf anderen Thread
- Simultaneous Multi-Threading (SMT): Im selben Takt werden Instruktionen von mehreren Threads gleichzeitig ausgegeben; Funktioniert auf superskalaren Prozessoren mit ungenutzten Slots.

Verbindungsnetzwerke

Verbindungsnetzwerk verbindet Komponenten eines Parallelrechners: Prozessoren mit Speichermodule, Ein-/Ausgabe-Geräte

- Statisches Netzwerk: Feste Punkt-zu-Punkt-Verbindungen, modelliert als Graph $G = (V, E)$
- Dynamisches Netzwerk: Verbindungen werden bei Bedarf durch Schaltelemente hergestellt

=> Bewertungskriterien:

Durchmesser δ	Maximale Distanz zwischen zwei Knoten
Grad g	Maximale Kantenanzahl an einem Knoten
Bisektionsband-breite B	Minimaler Kantenschnitt für gleich groß Hälften
Knotenkonnektivität nc	Minimale Knotenzahl deren Entfernung den Graph zerlegt

k-ary n-Cube

↳ n Dimensionen, k Prozessoren pro Dimension, $\rightarrow k^n$ Knoten

- k-ary 1-Cube: Ring der Länge k
- 2-ary n-Cube: Hypercube der Dimension n
- k-ary 2-Cube: 2D-Torus

- Kennwerte:

Ring Länge n	$\delta = \lfloor \frac{n}{2} \rfloor$, $g = 2$, $nc = 2$
d-dim. Mesh	Für $n_1 \times \dots \times n_d$: $\delta = \sum_i (n_i - 1)$, $g = 2d$, $nc = d$
d-dim. Torus	d-dim. Mesh mit Ränder verbunden => δ halbiert
k-dim. Hypercube	$n = 2^k$, $\delta = k$, $nc = k$, $B = \frac{n}{2}$

Streckungsgrad & Einbettung

- Einbettung: Abbildung von Netz NW1 auf Netz NW2 mit eindeutiger Zuordnung der Knoten
- Streckungsgrad: Maximaler Faktor, um den sich Distanzen erhöhen. Streckungsgrad = 1 => nachbarschafts-erhaltend

Klassifikation - Dynamische Netzwerke

blocking	Manche Permutationen nicht gleichzeitig schaltbar
non-blocking	Jede Permutation gleichzeitig Schaltbar
rearrangeable	Jede Permutation schaltbar, ggf. mit Routing

=> Crossbar: N^2 Schaltpunkte

Omega/Banyan: $(N/2) \cdot \log_2(N)$ Beta-Zellen

Künstliche Neuronale Netze

Künstliches Neuron

Einfachste Recheneinheit eines kNN; Nimmt mehrere Eingangssignale entgegen, gewichtet sie, summiert sie, vergleicht mit Schwellenwert, gibt Ausgangssignal aus.

- Komponenten:

Symbol	Bedeutung
x_i	Eingangsnummer i
w_i	Gewicht der Verbindungen von x_i zum Neuron
$net = \sum_i w_i \cdot x_i$	Netzeingabe
Θ	Schwellwert
f_act	Aktivierungsfunktion
o	Ausgabe (output)
t	Soll-Ausgabe (target) -> Training

McCulloch-Pitts-Neuron

[MP-Neuron]: Erstes mathematisches Modell eines Neurons; Binäres Schaltelement mit festem Schwellwert; Eingänge sind exzitatorisch (anregend) oder inhibitorisch (hemmend - eine einzige aktive inhibitorische Eingabe blockiert die Aktivierung

SLP vs. MLP

- Single Layer Perceptron (SLP): Eine Schicht zwischen Eingabe und Ausgabe; Nur linear trennbare Funktionen lernbar
- Multi Layer Perceptron (MLP): Min. 1 Hidden Layer; Approximiert beliebige Funktionen

Lernmethoden

- Überwachtes Lernen: Trainingsdaten enthalten Eingaben und Soll-Ausgaben
  - ↳ Algorithmen: Perceptron-Lernregel, Backpropagation
- Unüberwachtes Lernen: Nur Eingaben
  - ↳ Algorithmen: Self-Organizing Maps, Adaptive Resonance Theory
- Bestärkendes Lernen: Belohnung/Bestrafung
  - ↳ Algoritmen: Q-Learning, Deep-Q-Network, PPO

=> Shallow Networks: 1-2 verdeckte Schichten; Manuelle Feature-Extraktion, schnell trainiert

=> Deep Networks: Viele Schichten (>3), Hierarchische Features automatisch

CISC/RISC, SPRS

Complex Instruction Set Computer (CISC)

Architektur mit großem, komplexem Befehlssatz; Variable Befehlslänge, viele Adressierungsarten, mikroprogrammierte Steuerung

Reduced Instruction Set Computer (RISC)

Architektur mit kleinem, einfachem Befehlssatz; Einheitliche Befehlslänge, Load/Store-Architektur, festverdrahtete Steuerung

CPU-Time

CPU time = IC · CPI · T_C

↳ IC: Instruction Count (RISC: höher; CISC: niedriger)

↳ CPI: Cycles per Instruction (RISC: ≈1; CISC: >5)

↳ T_C: Clock Cycle Time (RISC niedriger durch verdrahtete Steuerung)

Spezielle Rechnerstrukturen

- Anwendungsspezifisch optimiert
- Höhere Leistung pro Watt für Spezialaufgaben
- GPU (Grafik), DSP (Signalverarbeitung, FPGA (rekonfigurierbar)